Neural design: mạch neural dấu chấm tĩnh cho vòng kín điều khiển mô hình DMN–Φ–QoC, mô hình hoá bằng VHDL

Báo cáo kỹ thuật – dự án `bqi_project`, thư mục `hw/` – Ngày 15 tháng 10 năm 2026

Tóm tắt. Bộ điều khiển vòng kín của bài Q1 (bộ lọc Kalman + LQG trên mô hình M1) được viết lại thành một *mạch neural* 3 lớp, 6 nơ-ron tuyến tính có làm tròn và bão hoà, hiện thực bằng VHDL-2008. Quét độ rộng từ cho thấy định dạng dấu chấm tĩnh **Q4.8 (13 bit)** là nhỏ nhất mà chi phí vòng kín chỉ cao hơn dấu chấm động **0,76 %**. Mạch được mô phỏng bằng GHDL 6.0.0: **270/270 epoch khớp từng bit** với bản tham chiếu Python, cả ở mức RTL lẫn **sau tổng hợp** (netlist cổng logic do GHDL sinh ra). Netlist có 16 bộ nhân hằng số khác 0, 24 bộ cộng và 107 bit thanh ghi; một epoch 10 s cần 4 chu kỳ xung nhịp. Chưa làm: tổng hợp lên FPGA cụ thể (Vivado/Quartus), kiểm thử nhánh bão hoà, và thay phương trình DMN bằng mạng neural học từ dữ liệu.

Mục lục

1 Mục tiêu và vị trí trong dự án **1**

2 Từ phương trình đến mạch neural **1**

3 Định dạng số và nơ-ron phần cứng **2**
3.1 Định dạng Q4.8 . . . 2
3.2 Phép tính của một nơ-ron . . . 2

4 Kiến trúc mạch tích hợp **3**
4.1 Mạch trong hệ thống neurofeedback . . . 3
4.2 Cổng vào/ra của `dmn_lqg_core` . . . 3
4.3 Cấu trúc RTL bên trong . . . 4
4.4 Máy trạng thái và định thời . . . 4

5 Cấu trúc mã VHDL và luồng thiết kế **5**

6 Chọn độ rộng từ **5**

7 Kiểm chứng **6**
7.1 Ba mức kiểm chứng . . . 6
7.2 Dạng sóng . . . 7
7.3 Một phiên 45 phút . . . 8

8 Tài nguyên sau tổng hợp **8**

9 Cách chạy lại **8**

10 Hạn chế và việc tiếp theo **9**

A Mã nguồn VHDL **9**

1 Mục tiêu và vị trí trong dự án

Thầy gợi ý "kết hợp với neural design". Báo cáo này làm theo cách hiểu *thiết kế mạng neural dạng mạch, mô hình hoá bằng VHDL*: lấy bộ điều khiển neurofeedback đã được kiểm chứng trong bài Q1 và biến nó thành một mạch số có thể nạp lên FPGA, chạy độc lập, trễ cỡ nano giây.

- **Nguồn gốc trong bài BQI gốc**: Thuật toán 2 (COM), dòng 5, $u_{\text{stim}} \leftarrow K(\text{QoC}_{\text{target}} - \text{QoC}(t))$, và bài toán điều khiển pt. 69–75.
- **Trong bài Q1**: viết lại thành LQG trên mô hình M1 (Gói 4, `model/control.py`); LQG thắng bang-bang 14 lần, cổng M2 đạt.
- **Trong báo cáo này**: hiện thực phần cứng của đúng bộ LQG đó, cùng ma trận $F, B, \mathbf{c}, L, K$.

Cách hiểu thứ hai (*thay phương trình DMN bằng mạng neural học từ dữ liệu*) chưa làm; xem mục 10.

2 Từ phương trình đến mạch neural

Mô hình M1 rời rạc hoá chính xác với epoch $\Delta = 10\,\text{s}$, trạng thái $\mathbf{x} = (A, \Phi)$:

$$\mathbf{x}_{k+1} = F\mathbf{x}_k + Bu_k + \mathbf{c} + \mathbf{w}_k, \qquad \mathbf{y}_k = \mathbf{x}_k + \boldsymbol{\eta}_k, \qquad \text{QoC}_k = \mathbf{h}^\top \mathbf{x}_k + \gamma_0.$$

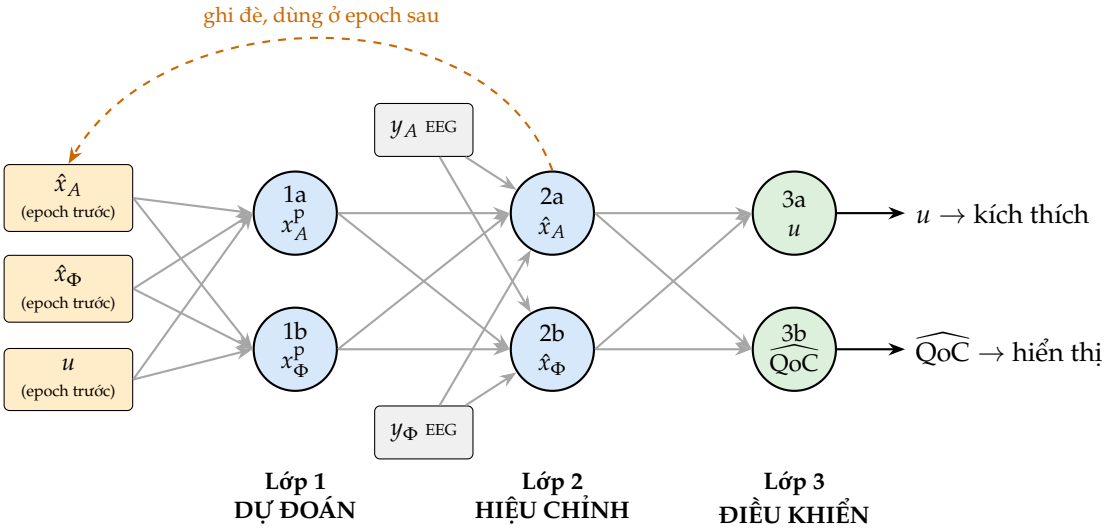
Bộ lọc Kalman dừng và luật LQG của Gói 4 là các phép biến đổi tuyến tính:

$$\begin{aligned} &\text{dự đoán:} && \mathbf{x}^{\text{pred}} = F\hat{\mathbf{x}} + Bu + \mathbf{c}, \\ &\text{hiệu chỉnh:} && \hat{\mathbf{x}} \leftarrow (I - L)\,\mathbf{x}^{\text{pred}} + L\,\mathbf{y}, \\ &\text{điều khiển:} && u = \text{clip}\big(-K\hat{\mathbf{x}} + (u_{\text{ss}} + K\mathbf{x}_{\text{ss}}),\ \pm u_{\max}\big), \qquad \widehat{\text{QoC}} = \mathbf{h}^\top \hat{\mathbf{x}} + \gamma_0. \end{aligned}$$

Mỗi dòng là một **nơ-ron tuyến tính**: tổng có trọng số các đầu vào cộng độ lệch. "Hàm kích hoạt" duy nhất là làm tròn và bão hoà về độ rộng từ, cộng thêm phép cắt $\pm u_{\max}$ ở nơ-ron điều khiển. Trọng số tính sẵn từ mô hình (không học), lượng tử hoá $q = \text{round}(v \cdot 2^8)$ (bảng 1).

Bảng 1: Trọng số và độ lệch của 6 nơ-ron: giá trị thực và số nguyên Q4.8 (giá trị thực $\approx q/256$). Tham số danh nghĩa M1, mục tiêu QoC $= 2{,}0$, $u_{\max} = 2$.

Nơ-ron	Đầu vào	Trọng số thực	Trọng số Q4.8 / độ lệch Q4.8
1a x_A^{pred}	$\hat{x}_A, \hat{x}_\Phi, u$	0,9672; 0; −0,1639	(248, 0, −42) / 5
1b x_Φ^{pred}	$\hat{x}_A, \hat{x}_\Phi, u$	−0,0132; 0,9943; 0,0011	(−3, 255, 0) / 3
2a $\hat{x}_A$	$x_A^{\text{pred}}, x_\Phi^{\text{pred}}, y_A, y_\Phi$	0,6884; 0,0063; 0,3116; −0,0063	(176, 2, 80, −2) / 0
2b $\hat{x}_\Phi$	$x_A^{\text{pred}}, x_\Phi^{\text{pred}}, y_A, y_\Phi$	0,0063; 0,8199; −0,0063; 0,1801	(2, 210, −2, 46) / 0
3a u	$\hat{x}_A, \hat{x}_\Phi$	3,6112; −4,5971	(924, −1177) / 1020
3b $\widehat{\text{QoC}}$	$\hat{x}_A, \hat{x}_\Phi$	−1,87; 2,34	(−479, 599) / 0
Reset: $\hat{\mathbf{x}} = \boldsymbol{\mu} = (0{,}600;\ 0{,}953) \to (154, 244)$; $u_{\max} = 2 \to 512$; độ lệch 1a, 1b $= \mathbf{c} = (0{,}0197;\ 0{,}0133)$; độ lệch 3a $= u_{\text{ss}} + K\mathbf{x}_{\text{ss}} = 3{,}983$.			



Hình 1: Mạch neural 3 lớp, 6 nơ-ron. Ô cam: thanh ghi giữ $\hat{x}_A, \hat{x}_\Phi, u$ của epoch trước; ô xám: đầu vào EEG của epoch hiện tại; vòng tròn xanh: nơ-ron tuyến tính (nhân hằng số, cộng, làm tròn, bão hoà); xanh lá: nơ-ron đầu ra. Kết quả của 2a, 2b và 3a ghi đè lên ba thanh ghi bên trái để dùng ở epoch sau (mũi tên đứt nét, vẽ đại diện cho cả ba).

3 Định dạng số và nơ-ron phần cứng

3.1 Định dạng Q4.8

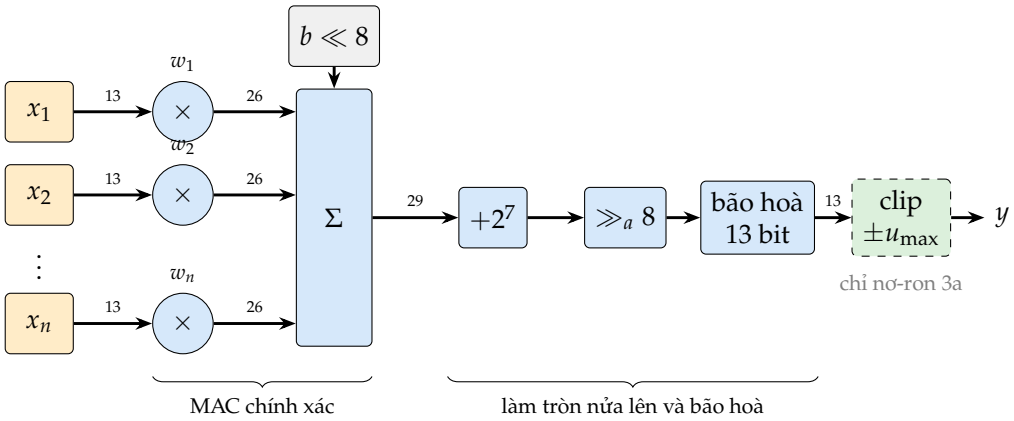
Số có dấu bù hai, $W = \text{FRAC} + 4 + 1 = 13$ bit: 1 bit dấu, 4 bit phần nguyên (dải $[-16, 16)$), 8 bit phần lẻ (bước $1/256 \approx 0{,}0039$). Dải ± 16 đủ chứa trọng số lớn nhất $|K_\Phi| = 4{,}6$ và độ lệch lớp 3 là 3,98.

3.2 Phép tính của một nơ-ron

Với đầu vào x_i và trọng số w_i là số nguyên Q4.8:

$$\text{acc} = (b \ll 8) + \sum_i w_i x_i \quad \text{(chính xác, } 2W + 3 = 29 \text{ bit)}, \qquad y = \text{sat}_{13}\big((\text{acc} + 2^7) \gg_a 8\big),$$

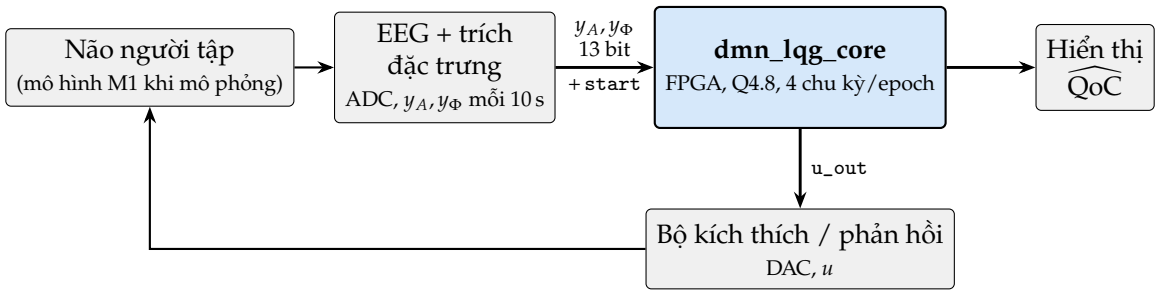
tức nhân chính xác, cộng nửa LSB để làm tròn, dịch phải số học 8 bit, rồi bão hoà về 13 bit. Bản Python (`model/hardware.py`, hàm `neuron`) và bản VHDL (`neuron_pkg.vhd`, hàm `neuron`) cài đúng cùng ngữ nghĩa này; đó là điều kiện để hai bên khớp từng bit.



Hình 2: Đường dữ liệu của một nơ-ron (hàm `neuron` trong `neuron_pkg.vhd`). Trọng số là hằng số, nên mỗi "×" là một bộ nhân với hằng số; số trên mũi tên là độ rộng bus (bit).

4 Kiến trúc mạch tích hợp

4.1 Mạch trong hệ thống neurofeedback



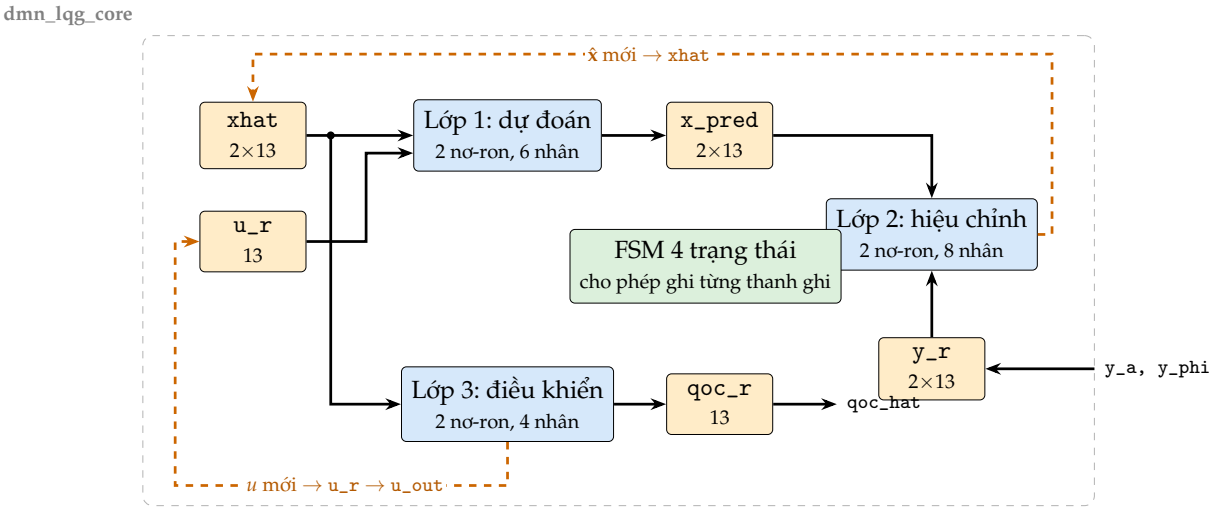
Hình 3: Vị trí của mạch trong vòng neurofeedback. Báo cáo này thiết kế khối giữa; các khối xám là giao tiếp ngoài (chưa thiết kế).

4.2 Cổng vào/ra của `dmn_lqg_core`

Bảng 2: Thực thể `dmn_lqg_core` (VHDL-2008).

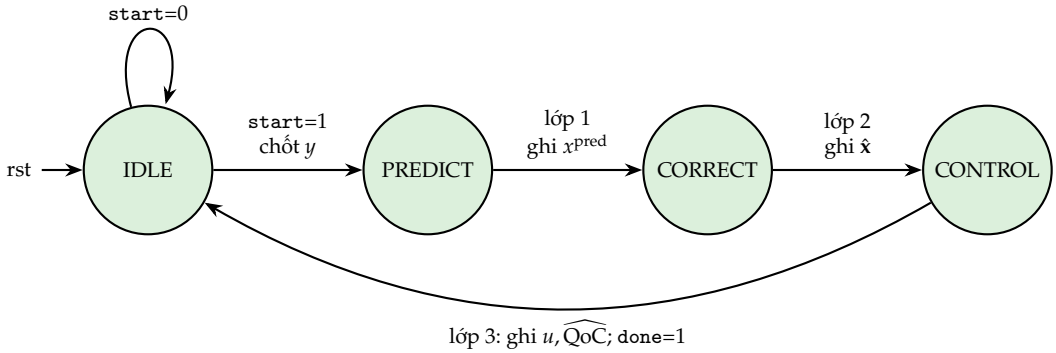
Cổng	Hướng	Kiểu	Ý nghĩa
`clk`	in	std_logic	xung nhịp
`rst`	in	std_logic	reset đồng bộ, mức cao: $\hat{\mathbf{x}} = \boldsymbol{\mu}$, $u = \text{control}(\boldsymbol{\mu})$
`start`	in	std_logic	xung 1 chu kỳ khi có mẫu EEG mới (mỗi epoch)
`y_a, y_phi`	in	signed(12:0)	đặc trưng EEG của A_{DMN} và Φ, Q4.8
`u_out`	out	signed(12:0)	tín hiệu điều khiển cho epoch sau
`xhat_a, xhat_phi`	out	signed(12:0)	trạng thái ước lượng
`qoc_hat`	out	signed(12:0)	QoC ước lượng (giám sát)
`done`	out	std_logic	bật 1 chu kỳ khi các đầu ra hợp lệ

4.3 Cấu trúc RTL bên trong



Hình 4: Cấu trúc RTL. Ô cam: thanh ghi (tổng 107 bit); ô xanh: logic tổ hợp của các nơ-ron; ô xanh lá: máy trạng thái điều khiển thanh ghi nào được ghi ở chu kỳ nào. Đường đứt nét: kết quả lớp 2 và lớp 3 ghi đè lên `xhat` và `u_r`, dùng lại ở epoch sau.

4.4 Máy trạng thái và định thời



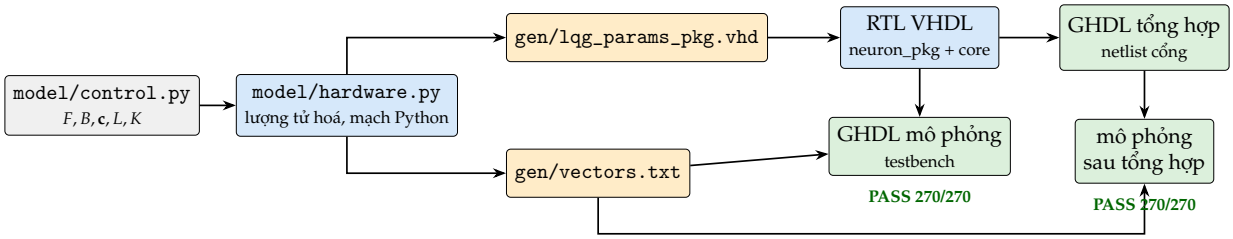
Hình 5: Máy trạng thái của `dmn_lqg_core`: mỗi trạng thái tính một lớp, kết quả ghi vào thanh ghi ở sườn lên kế tiếp.

Từ khi `start` được lấy mẫu đến khi `done` bật là **4 chu kỳ** (chốt y, lớp 1, lớp 2, lớp 3). Ở 100 MHz đó là 40 ns, nhỏ hơn epoch 10 s khoảng $2{,}5 \times 10^8$ lần: mạch rảnh gần như toàn bộ thời gian, nên có thể hạ xung nhịp hoặc dùng chung bộ nhân để tiết kiệm tài nguyên (mục 10).

5 Cấu trúc mã VHDL và luồng thiết kế

Bảng 3: Các file của phần neural design.

File	Vai trò
`model/hardware.py`	Bản tham chiếu Python *bit-exact*; quét số bit; sinh hai file trong `gen/`
`hw/vhdl/gen/lqg_params_pkg.vhd`	*Sinh tự động*: `W`, `FRAC`, mọi trọng số và độ lệch nguyên (listing 1)
`hw/vhdl/neuron_pkg.vhd`	Kiểu `fx_t`; hàm `neuron`, `saturate`, `clip` (listing 2)
`hw/vhdl/dmn_lqg_core.vhd`	Thực thể và kiến trúc RTL: thanh ghi, FSM, 6 lời gọi `neuron` (listing 3)
`hw/vhdl/tb_dmn_lqg_core.vhd`	Testbench: đọc vectơ, đưa vào từng epoch, so 4 đầu ra (listing 4)
`hw/vhdl/gen/vectors.txt`	*Sinh tự động*: 270 epoch $(y_A, y_\Phi \mid \hat{x}_A, \hat{x}_\Phi, u, \widehat{\mathrm{QoC}})$
`hw/run_ghdl.ps1, .sh`	Phân tích, elaboration, mô phỏng; `-Wave` xuất dạng sóng
`hw/report/make_assets.py`	Tổng hợp GHDL, thống kê netlist, mô phỏng sau tổng hợp, hình báo cáo
`tests/test_hardware.py`	5 test Python



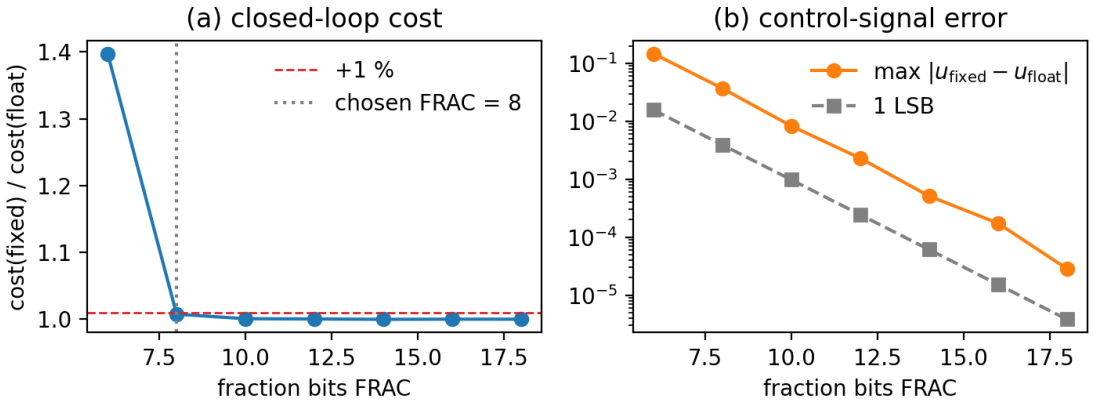
Hình 6: Luồng thiết kế. Hằng số và dữ liệu kiểm tra luôn sinh từ cùng một mã Python, nên đổi mô hình hay đổi số bit chỉ cần chạy lại `python main.py model hardware`.

6 Chọn độ rộng từ

Đối tượng (não, mô hình M1) giữ dấu chấm động; chỉ bộ điều khiển bị lượng tử hoá. Mỗi định dạng chạy 20 phiên $\times$ 270 epoch với cùng nhiễu như bộ điều khiển dấu chấm động. Quy tắc chọn: FRAC nhỏ nhất có chi phí vòng kín vượt dấu chấm động không quá 1 %.

Bảng 4: Quét số bit phần lẻ (`outputs/results/model/hardware_results.json`).

FRAC	W (bit)	Chi phí dấu chấm tĩnh	Chi phí dấu chấm động	Tỉ số	$\max \lvert u_{\text{fixed}} - u_{\text{float}} \rvert$
6	11	0,05507	0,03942	1,397	$1{,}5 \times 10^{-1}$
8	**13**	**0,03972**	0,03942	**1,0076**	$3{,}7 \times 10^{-2}$
10	15	0,03945	0,03942	1,0007	$8{,}3 \times 10^{-3}$
12	17	0,03943	0,03942	1,0004	$2{,}3 \times 10^{-3}$
14	19	0,03941	0,03942	0,9998	$5{,}1 \times 10^{-4}$
16	21	0,03942	0,03942	1,0001	$1{,}7 \times 10^{-4}$
18	23	0,03942	0,03942	1,0000	$2{,}9 \times 10^{-5}$



Hình 7: (a) Tỉ số chi phí theo số bit phần lẻ: dưới 8 bit vòng kín suy giảm nhanh, từ 10 bit không còn khác biệt. (b) Sai lệch lớn nhất của tín hiệu điều khiển, giảm khoảng 4 lần mỗi khi thêm 2 bit, song song với 1 LSB.

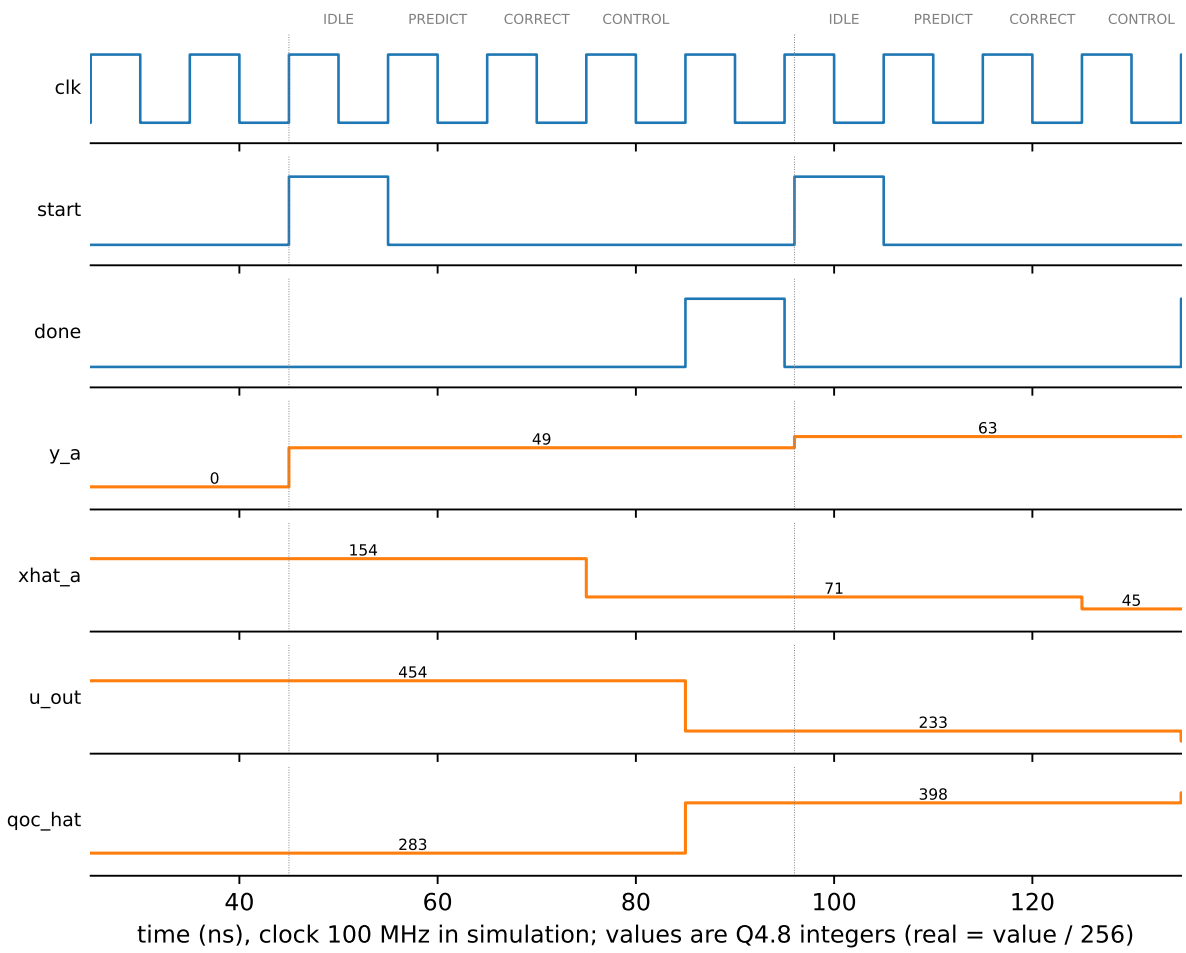
7 Kiểm chứng

7.1 Ba mức kiểm chứng

Bảng 5: Kết quả kiểm chứng (GHDL 6.0.0, VHDL-2008, `hw/report/assets/synth_stats.json`).

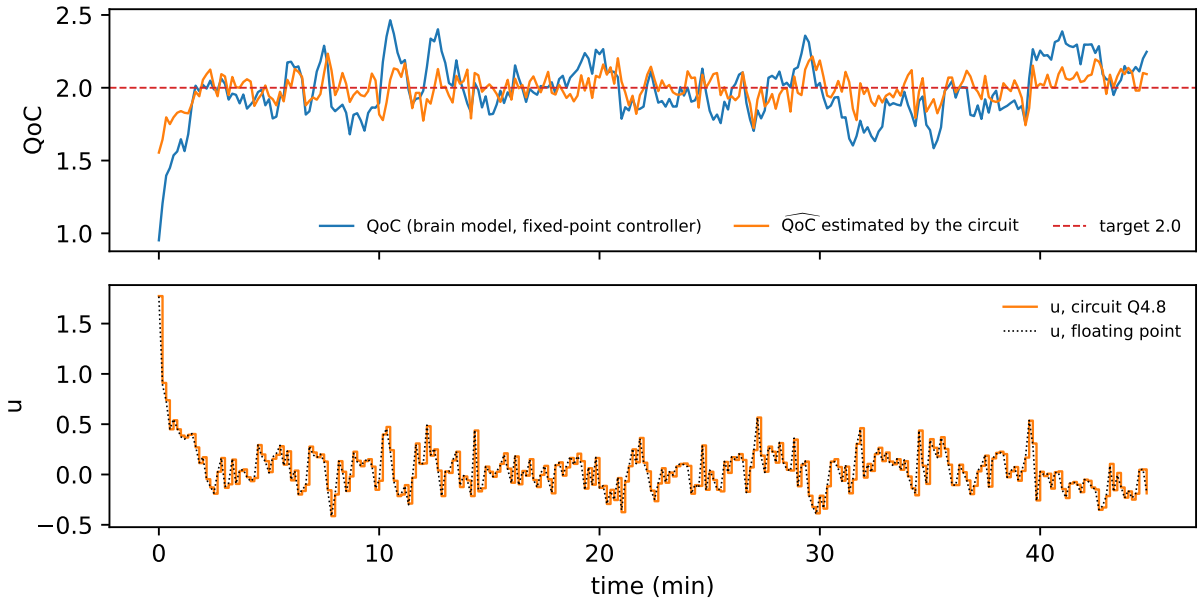
Mức	Cách làm	Kết quả
Python	5 test: làm tròn/bão hoà; đường dấu chấm động trùng `simulate_policy` của Gói 4 tới 10^{-12}; tất định; file sinh ra hợp lệ	5/5 đạt
RTL	Testbench đưa 270 epoch y, so $\hat{x}_A, \hat{x}_\Phi, u, \widehat{\text{QoC}}$ với Python	**270/270 khớp từng bit**
Sau tổng hợp	Cùng testbench trên netlist cổng logic do `ghdl -synth` sinh	**270/270 khớp từng bit**
Kiểm tra ngược	Sửa sai 1 LSB của u ở epoch 100	báo đúng epoch 100, `FAIL 1/270`

7.2 Dạng sóng



Hình 8: Hai epoch đầu của mô phỏng RTL (đọc từ file VCD). Trước epoch đầu, các giá trị là giá trị reset: $\hat{x}_A = 154$ (0,60 = mức nền DMN), $\widehat{\mathrm{QoC}} = 283$ (1,105 $\approx \mathrm{QoC}^*$), $u = 454$ (1,77). Mỗi epoch: `start` được lấy mẫu, sau 2 chu kỳ `xhat_a` đổi, sau 3 chu kỳ `u_out`, `qoc_hat` đổi và `done` bật.

7.3 Một phiên 45 phút



Hình 9: Phiên seed 42, mạch Q4.8 điều khiển mô hình M1. Trên: QoC thật của mô hình và QoC mạch ước lượng, dao động quanh mục tiêu 2,0. Dưới: tín hiệu điều khiển của mạch (bậc thang Q4.8) gần như trùng bộ điều khiển dấu chấm động; sai lệch lớn nhất 0,025. Chi phí phiên: 0,04600 (mạch) so với 0,04577 (dấu chấm động).

8 Tài nguyên sau tổng hợp

Lệnh `ghdl --synth` chuyển RTL thành netlist cổng logic dạng VHDL, dài 870 dòng, lưu tại `hw/vhdl/work/netlist_synth.vhd`. Đếm trực tiếp trên netlist:

Bảng 6: Thành phần của netlist sau tổng hợp (GHDL, chưa ánh xạ lên FPGA cụ thể).

Thành phần	Số lượng	Ghi chú
Bộ nhân với hằng số	18	khớp thiết kế $6+8+4$; 2 bộ nhân với 0 (trọng số 0 trong lớp 1) sẽ bị công cụ FPGA loại
trong đó khác 0	16	13 bit $\times$ hằng số; ánh xạ được sang DSP hoặc mạch dịch–cộng
Bộ cộng / trừ	24 / 6	cây cộng của các nơ-ron, làm tròn
Thanh ghi (bit)	107	y: 26, x^{pred}: 26, $\hat{\mathbf{x}}$: 26, u: 13, $\widehat{\text{QoC}}$: 13, FSM: 2, `done`: 1
+ bit giám sát	6	do các lệnh `assert` trong `neuron`; hằng 1, bị FPGA loại

Đây là số đếm từ netlist chung, **chưa phải** báo cáo tài nguyên của một chip cụ thể (LUT, FF, DSP, tần số tối đa). Con số đó cần Vivado hoặc Quartus, chưa cài trên máy.

9 Cách chạy lại

Trong thư mục gốc `K:\bqi_project`:

1. `python main.py model hardware` – quét số bit, sinh hai file trong `hw/vhdl/gen/`.
2. `python -m pytest tests/test_hardware.py` – 5 test Python.
3. `powershell -ExecutionPolicy Bypass -File hw\run_ghdl.ps1` – mô phỏng RTL, kết quả PASS 270/270; thêm `-Wave` để xuất dạng sóng `.vcd` / `.ghw` (xem bằng Surfer).

4. `python hw\report\make_assets.py` – tổng hợp, mô phỏng sau tổng hợp, sinh hình của báo cáo.
5. Trong `hw\report`: `pdflatex bao_cao_neural_design.tex` (chạy 2 lần).

10 Hạn chế và việc tiếp theo

1. **Nhánh bão hoà chưa được kiểm thử.** Trong phiên thử, $u \in [-0{,}41;\ 1{,}77]$ chưa chạm ± 2, và không giá trị nào chạm biên 13 bit. Cần thêm bộ vectơ có đầu vào cực đoan.
2. **Chưa tổng hợp lên FPGA cụ thể**: chưa có số LUT/FF/DSP và tần số tối đa.
3. **Phiên bản tiết kiệm**: một bộ nhân dùng chung (khoảng 18 chu kỳ/epoch thay vì 4) để so tài nguyên.
4. **Mô hình não bằng VHDL** (M1 + nhiễu LFSR) để cả vòng kín chạy trên FPGA.
5. **Độ bền kết hợp**: bộ điều khiển thiết kế trên tham số *ước lượng* (24 ô của Hình 7 bài Q1) *và* lượng tử hoá Q4.8.
6. **Thay phương trình DMN bằng mạng neural học từ dữ liệu** (cách hiểu thứ hai của "neural design"). Các nơ-ron hiện tại tuyến tính, trọng số tính sẵn từ phương trình. Không có dữ liệu thật thì mạng neural chỉ học lại M1; hướng có ý nghĩa là sinh dữ liệu từ một "não giả" phi tuyến, fit cả M1 và một mạng neural nhỏ, so độ chính xác dự đoán với khả năng xác định tham số, rồi đưa mạng học được sang VHDL bằng chính khung mạch này. Cần thầy xác nhận hướng.

A Mã nguồn VHDL

Đoạn mã 1: `gen/lqg_params_pkg.vhd` (sinh tự động)

```
-- Generated by model/hardware.py -- DO NOT EDIT (python main.py model hardware)
-- Fixed-point format Q4.8 signed, W = 13 bits; nominal M1 parameters, seed 42.
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

package lqg_params_pkg is
  constant W    : natural := 13;
  constant FRAC : natural := 8;
  type int_vec is array (natural range <>) of integer;
  -- layer 1: predict   x_pred_i = W1_i . (xhat_A, xhat_Phi, u) + b1_i
  constant W1_A : int_vec(0 to 2) := (248, 0, -42);
  constant W1_PHI : int_vec(0 to 2) := (-3, 255, 0);
  constant B1_A   : integer := 5;
  constant B1_PHI : integer := 3;
  -- layer 2: correct   xhat_i = W2_i . (x_pred_A, x_pred_Phi, y_A, y_Phi) + b2_i
  constant W2_A : int_vec(0 to 3) := (176, 2, 80, -2);
  constant W2_PHI : int_vec(0 to 3) := (2, 210, -2, 46);
  constant B2_A   : integer := 0;
  constant B2_PHI : integer := 0;
  -- layer 3: control   u = clip(W3 . xhat + B3, +-UMAX_Q);  read-out QoC^ = W4 . xhat + B4
  constant W3 : int_vec(0 to 1) := (924, -1177);
  constant B3     : integer := 1020;
  constant W4 : int_vec(0 to 1) := (-479, 599);
  constant B4     : integer := 0;
  constant UMAX_Q : integer := 512;
  -- reset state: stationary mean of M1
  constant MU_Q : int_vec(0 to 1) := (154, 244);
```

```
  -- float values for reference
  -- F = [[ 0.967216,  0.      ],  [-0.01324 ,  0.994349]]
  -- B = [-0.163919,  0.00111 ], c = [0.01967 , 0.013329]
  -- L = [[ 0.311593, -0.006267],  [-0.006267,  0.180071]]
  -- K = [-3.611248,  4.597133], x_ss = [0.480168, 1.238424], u_ss = 0.023966
end package lqg_params_pkg;
```

Đoạn mã 2: `neuron_pkg.vhd`

```
-- ============================================================================
--  neuron_pkg : linear neuron with rounding and saturation (fixed point)
--
--  Reference semantics = model/hardware.py :: neuron()
--      acc = (bias << FRAC) + sum_i w_i * x_i          exact, ACC_W bits
--      y   = saturate_W( (acc + 2^(FRAC-1)) >>a FRAC )  round half up, arithmetic shift
--  W and FRAC come from the generated package lqg_params_pkg.
-- ============================================================================
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
use work.lqg_params_pkg.all;

package neuron_pkg is
  constant ACC_W : natural := 2 * W + 3;
  subtype fx_t is signed(W - 1 downto 0);
  type fx_vec is array (natural range <>) of fx_t;

  function saturate(a : signed) return fx_t;
  function neuron(x : fx_vec; wt : int_vec; bias : integer) return fx_t;
  function clip(x : fx_t; lim : integer) return fx_t;
end package neuron_pkg;

package body neuron_pkg is

  function saturate(a : signed) return fx_t is
    constant HI : signed(a'length - 1 downto 0) := to_signed(2 ** (W - 1) - 1, a'length);
    constant LO : signed(a'length - 1 downto 0) := to_signed(-(2 ** (W - 1)), a'length);
  begin
    if a > HI then
      return resize(HI, W);
    elsif a < LO then
      return resize(LO, W);
    else
      return resize(a, W);
    end if;
  end function;

  -- NB: VHDL identifiers are case-insensitive, so the weight parameter must not
  -- be called "w" (it would hide the word-width constant W).
  function neuron(x : fx_vec; wt : int_vec; bias : integer) return fx_t is
    variable acc : signed(ACC_W - 1 downto 0);
  begin
    assert x'length = wt'length report "neuron: input/weight length mismatch" severity failure;
    acc := shift_left(to_signed(bias, ACC_W), FRAC);
    for i in 0 to x'length - 1 loop
      acc := acc + resize(x(x'low + i) * to_signed(wt(wt'low + i), W), ACC_W);
    end loop;
    acc := acc + to_signed(2 ** (FRAC - 1), ACC_W);
    acc := shift_right(acc, FRAC);          -- arithmetic shift on signed
    return saturate(acc);
  end function;

  function clip(x : fx_t; lim : integer) return fx_t is
  begin
    if x > to_signed(lim, W) then
      return to_signed(lim, W);
    elsif x < to_signed(-lim, W) then
```

```
      return to_signed(-lim, W);
    else
      return x;
    end if;
  end function;

end package body neuron_pkg;
```

Đoạn mã 3: `dmn_lqg_core.vhd`

```
-- ============================================================================
--  dmn_lqg_core : the M1 closed loop (stationary Kalman filter + LQG regulator)
--  as a three-layer neural circuit, one epoch (10 s) per 'start' pulse.
--
--      layer 1  PREDICT  x_pred = F xhat + B u + c            2 neurons, 3 inputs
--      layer 2  CORRECT  xhat   = (I-L) x_pred + L y          2 neurons, 4 inputs
--      layer 3  CONTROL  u      = clip(-K xhat + b3, +-u_max) 1 neuron
--               READOUT  qoc    = h xhat + gamma0             1 neuron (monitor)
--
--  Timing: 3 clock cycles per epoch (one layer per cycle), 'done' is high for
--  one cycle when u / xhat / qoc_hat are valid.  On reset the state is the
--  stationary mean MU_Q and u = control(MU_Q), exactly as NeuralCircuit.reset()
--  in model/hardware.py.  All constants come from gen/lqg_params_pkg.vhd.
-- ============================================================================
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
use work.lqg_params_pkg.all;
use work.neuron_pkg.all;

entity dmn_lqg_core is
  port (
    clk      : in  std_logic;
    rst      : in  std_logic;                      -- synchronous, active high
    start    : in  std_logic;                      -- one pulse per epoch with new y
    y_a      : in  signed(W - 1 downto 0);         -- EEG feature for A_DMN (Q format)
    y_phi    : in  signed(W - 1 downto 0);         -- EEG feature for Phi
    u_out    : out signed(W - 1 downto 0);         -- control input for the next epoch
    xhat_a   : out signed(W - 1 downto 0);
    xhat_phi : out signed(W - 1 downto 0);
    qoc_hat  : out signed(W - 1 downto 0);         -- estimated QoC (monitoring)
    done     : out std_logic
  );
end entity dmn_lqg_core;

architecture rtl of dmn_lqg_core is
  type state_t is (IDLE, PREDICT, CORRECT, CONTROL);
  signal state   : state_t := IDLE;
  signal xhat    : fx_vec(0 to 1) := (to_signed(MU_Q(0), W), to_signed(MU_Q(1), W));
  signal x_pred  : fx_vec(0 to 1) := (others => (others => '0'));
  signal u_r     : fx_t := (others => '0');
  signal qoc_r   : fx_t := (others => '0');
  signal y_r     : fx_vec(0 to 1) := (others => (others => '0'));
  signal done_r  : std_logic := '0';

  constant MU_FX : fx_vec(0 to 1) := (to_signed(MU_Q(0), W), to_signed(MU_Q(1), W));
begin

  process (clk)
    variable in1 : fx_vec(0 to 2);
    variable in2 : fx_vec(0 to 3);
  begin
    if rising_edge(clk) then
      done_r <= '0';
      if rst = '1' then
        state  <= IDLE;
        xhat   <= MU_FX;
```

```
        u_r    <= clip(neuron(MU_FX, W3, B3), UMAX_Q);
        qoc_r  <= neuron(MU_FX, W4, B4);
      else
        case state is
          when IDLE =>
            if start = '1' then
              y_r(0) <= y_a;
              y_r(1) <= y_phi;
              state  <= PREDICT;
            end if;

          when PREDICT =>                          -- layer 1
            in1 := (xhat(0), xhat(1), u_r);
            x_pred(0) <= neuron(in1, W1_A, B1_A);
            x_pred(1) <= neuron(in1, W1_PHI, B1_PHI);
            state <= CORRECT;

          when CORRECT =>                          -- layer 2
            in2 := (x_pred(0), x_pred(1), y_r(0), y_r(1));
            xhat(0) <= neuron(in2, W2_A, B2_A);
            xhat(1) <= neuron(in2, W2_PHI, B2_PHI);
            state <= CONTROL;

          when CONTROL =>                          -- layer 3 + read-out
            u_r    <= clip(neuron(xhat, W3, B3), UMAX_Q);
            qoc_r  <= neuron(xhat, W4, B4);
            done_r <= '1';
            state  <= IDLE;
        end case;
      end if;
    end if;
  end process;

  u_out    <= u_r;
  xhat_a   <= xhat(0);
  xhat_phi <= xhat(1);
  qoc_hat  <= qoc_r;
  done     <= done_r;

end architecture rtl;
```

Đoạn mã 4: `tb_dmn_lqg_core.vhd`

```
-- ============================================================================
--  tb_dmn_lqg_core : bit-exact check of dmn_lqg_core against gen/vectors.txt
--  (produced by python main.py model hardware).
--
--  vectors.txt format:
--      # comment lines
--      N
--      y_a y_phi xhat_a xhat_phi u qoc     (N lines, integers in Q format)
--  Each line is one epoch: the testbench applies (y_a, y_phi), pulses 'start',
--  waits for 'done' and compares the four outputs with the expected integers.
--  Run:  ghdl -a --std=08 gen/lqg_params_pkg.vhd neuron_pkg.vhd dmn_lqg_core.vhd tb_dmn_lqg_core.vhd
--        ghdl -e --std=08 tb_dmn_lqg_core
--        ghdl -r --std=08 tb_dmn_lqg_core -gVEC_FILE=gen/vectors.txt
-- ============================================================================
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
use std.textio.all;
use work.lqg_params_pkg.all;
use work.neuron_pkg.all;

entity tb_dmn_lqg_core is
  generic (VEC_FILE : string := "gen/vectors.txt");
end entity tb_dmn_lqg_core;
```

```
25
26  architecture sim of tb_dmn_lqg_core is
27    constant T_CLK : time := 10 ns;
28    signal clk      : std_logic := '0';
29    signal rst      : std_logic := '1';
30    signal start    : std_logic := '0';
31    signal y_a      : signed(W - 1 downto 0) := (others => '0');
32    signal y_phi    : signed(W - 1 downto 0) := (others => '0');
33    signal u_out    : signed(W - 1 downto 0);
34    signal xhat_a   : signed(W - 1 downto 0);
35    signal xhat_phi : signed(W - 1 downto 0);
36    signal qoc_hat  : signed(W - 1 downto 0);
37    signal done     : std_logic;
38    signal finished : boolean := false;
39  begin
40
41    dut : entity work.dmn_lqg_core
42      port map (clk => clk, rst => rst, start => start, y_a => y_a, y_phi => y_phi,
43                u_out => u_out, xhat_a => xhat_a, xhat_phi => xhat_phi, qoc_hat => qoc_hat, done => done);
44
45    clk <= not clk after T_CLK / 2 when not finished else '0';
46
47    stimulus : process
48      file vec       : text;
49      variable l     : line;
50      variable ch    : character;
51      variable n     : integer;
52      variable v_ya, v_yp, e_xa, v_dummy : integer;
53      variable e_xp, e_u, e_q : integer;
54      variable errors : integer := 0;
55      variable k     : integer := 0;
56      variable good  : boolean;
57    begin
58      file_open(vec, VEC_FILE, read_mode);
59      -- skip comment lines, then read N
60      loop
61        readline(vec, l);
62        if l'length > 0 and l(1) /= '#' then
63          read(l, n);
64          exit;
65        end if;
66      end loop;
67      report "tb_dmn_lqg_core: " & integer'image(n) & " vectors, W=" & integer'image(W) &
68             " FRAC=" & integer'image(FRAC);
69
70      -- reset
71      rst <= '1';
72      wait for 3 * T_CLK;
73      wait until rising_edge(clk);
74      rst <= '0';
75      wait until rising_edge(clk);
76
77      for i in 1 to n loop
78        readline(vec, l);
79        read(l, v_ya);
80        read(l, v_yp);
81        read(l, e_xa);
82        read(l, e_xp);
83        read(l, e_u);
84        read(l, e_q);
85
86        y_a   <= to_signed(v_ya, W);
87        y_phi <= to_signed(v_yp, W);
88        start <= '1';
89        wait until rising_edge(clk);
90        start <= '0';
91        -- wait for done (3 cycles)
```

```
92        loop
93          wait until rising_edge(clk);
94          exit when done = '1';
95        end loop;
96        wait for 1 ns;    -- let outputs settle after the edge
97        good := (to_integer(xhat_a) = e_xa) and (to_integer(xhat_phi) = e_xp) and
98                (to_integer(u_out) = e_u) and (to_integer(qoc_hat) = e_q);
99        if not good then
100         errors := errors + 1;
101         if errors <= 10 then
102           report "MISMATCH epoch " & integer'image(i) &
103                  ": xhat=(" & integer'image(to_integer(xhat_a)) & "," & integer'image(to_integer(xhat_phi
      )) &
104                  ") u=" & integer'image(to_integer(u_out)) & " qoc=" & integer'image(to_integer(qoc_hat))
       &
105                  "  expected (" & integer'image(e_xa) & "," & integer'image(e_xp) & ") u=" &
106                  integer'image(e_u) & " qoc=" & integer'image(e_q)
107             severity warning;
108         end if;
109       end if;
110       k := k + 1;
111     end loop;
112     file_close(vec);
113
114     if errors = 0 then
115       report "PASS: " & integer'image(k) & " epochs bit-exact with model/hardware.py" severity note;
116     else
117       report "FAIL: " & integer'image(errors) & " of " & integer'image(k) & " epochs differ" severity
       error;
118     end if;
119     finished <= true;
120     wait;
121   end process;
122
123 end architecture sim;
```